

SQL Injection Attacks and Prevention Methods

Student: Oancea Teodora

Professor: Pop Emilia

I. INTRODUCTION

SQL Injection is a critical security vulnerability where an attacker supplies specially crafted input that alters the intended structure of a database query. By injecting malicious SQL into form fields or URL parameters, an attacker can bypass authentication, retrieve or modify sensitive data, or even delete entire tables. Preventing SQL injection requires treating user input as data, not executable code, through techniques like parameterized queries, input validation, and whitelisting. This document illustrates five common injection scenarios and shows how each can be mitigated.

1. Classic SQL Injection on Members (first_name)

Description:

This attack manipulates a query by injecting a payload into the WHERE clause, causing it to always evaluate true.

Vulnerable Query:

```
DECLARE @input1 NVARCHAR(50);
SET @input1 = 'Cristian' OR 1=1 --';
DECLARE @sql1 NVARCHAR(MAX);
SET @sql1 = 'SELECT * FROM Members WHERE first_name = '' + @input1
+ ''';
EXEC sp_executesql @sql1;
```

Effect: Returns all rows from the Members table because OR 1=1 is always true.

Prevention: Use parameterized queries (prepared statements) so that user input is treated strictly

```
DECLARE @input1_safe NVARCHAR(50) = 'Cristian' OR 1=1 --';
DECLARE @sql1_safe NVARCHAR(MAX) = N'
SELECT *
FROM Members
WHERE first_name = @firstname;
';
EXEC sp_executesql
@sql1_safe,
```

```
N'@firstname NVARCHAR(50)',  
@firstname = @input1_safe;
```

2. Union-Based Injection on Books (genre)

Description:

An attacker uses UNION SELECT to append attacker-controlled columns to the original query result set.

Vulnerable Query:

```
DECLARE @input2 NVARCHAR(100) = 'Fantasy';  
DECLARE @sql2 NVARCHAR(MAX) =  
    'SELECT title, author FROM Books WHERE genre = ''' + @input2 +  
    ''';';  
EXEC sp_executesql @sql2;  
-- Malicious extension:  
DECLARE @input2_mal NVARCHAR(100) =  
    ''' UNION SELECT CAST(members_id AS NVARCHAR(10)), email FROM  
Members --';  
DECLARE @sql2_mal NVARCHAR(MAX) =  
    'SELECT title, author FROM Books WHERE genre = ''' +  
    @input2_mal + ''';';  
EXEC sp_executesql @sql2_mal;
```

Effect: The UNION SELECT pulls members_id and email from the Members table and appends them to the Books result.

Prevention: Parameterize the genre input so that it cannot break out of the WHERE clause.
Example:

```
DECLARE @input2_safe NVARCHAR(100) =  
    ''' UNION SELECT CAST(members_id AS NVARCHAR(10)), email FROM  
Members --';  
  
DECLARE @sql2_safe NVARCHAR(MAX) = N'  
    SELECT title, author  
    FROM Books  
    WHERE genre = @genre;  
';  
  
EXEC sp_executesql
```

```
@sql2_safe,  
N'@genre NVARCHAR(100)',  
@genre = @input2_safe;
```

3. LIKE Clause Injection on Book_Requests (book_title)

Description: Injection into the LIKE pattern can bypass intended filtering logic.

Vulnerable Query:

```
DECLARE @input3 NVARCHAR(100) = 'The Hobbit';  
DECLARE @sql3 NVARCHAR(MAX) =  
    'SELECT * FROM Book_Requests WHERE book_title LIKE ''%' +  
    @input3 + '%''';  
EXEC sp_executesql @sql3;  
-- Malicious input:  
DECLARE @input3_mal NVARCHAR(100) = 'The Hobbit%' OR 1=1 --';  
DECLARE @sql3_mal NVARCHAR(MAX) =  
    'SELECT * FROM Book_Requests WHERE book_title LIKE ''%' +  
    @input3_mal + '%''';  
EXEC sp_executesql @sql3_mal;
```

Effect: The injected OR 1=1 causes WHERE book_title LIKE '%The Hobbit%' OR 1=1, returning all rows.

Prevention: Build the LIKE pattern using a parameter, ensuring the entire string is treated as data:

```
DECLARE @input3_safe NVARCHAR(100) = 'The Hobbit%' OR 1=1 --';  
DECLARE @pattern3 NVARCHAR(120) = '%' + @input3_safe + '%';  
  
DECLARE @sql3_safe NVARCHAR(MAX) = N'  
    SELECT *  
    FROM Book_Requests  
    WHERE book_title LIKE @searchTerm;  
';  
  
EXEC sp_executesql  
    @sql3_safe,  
    N'@searchTerm NVARCHAR(120)',  
    @searchTerm = @pattern3;
```

4. ORDER BY Clause Injection on Books (ordering by a column)

Description: Injection into the ORDER BY clause can allow execution of arbitrary SQL commands.

Vulnerable Query:

```
-- Suppose @sortCol comes from user input:
DECLARE @sortCol NVARCHAR(50) = 'title; DROP TABLE Book_Reviews; -
-';
DECLARE @sql4 NVARCHAR(MAX) = 'SELECT * FROM Books ORDER BY ' +
@sortCol + ' ';
EXEC sp_executesql @sql4;
```

Effect: If permissions allow, Book_Reviews table is dropped.

Prevention: Whitelist allowed column names on the server side and use QUOTENAME() to bracket the identifier. Reject any input not in the approved list. For example:

```
DECLARE @sortCol_safe NVARCHAR(50) = 'title; DROP TABLE
Book_Reviews; --';
IF @sortCol_safe NOT IN (N'book_id', N'title', N'author', N'genre',
N'published_year')
BEGIN
    RAISERROR('Invalid sort column', 16, 1);
    RETURN;
END

DECLARE @sql4_safe NVARCHAR(MAX) =
'SELECT * FROM Books ORDER BY ' + QUOTENAME(@sortCol_safe) +
' ';

EXEC sp_executesql @sql4_safe;
```

5. Stacked Queries Injection on Meetings (location_place)

Description: Injection of stacked queries allows execution of multiple commands in a single statement.

Vulnerable Query:

```
DECLARE @input5 NVARCHAR(100) = 'Library';
DECLARE @sql5 NVARCHAR(MAX) =
```

```

        'SELECT * FROM Meetings WHERE location_place = '' + @input5 +
        ''';';
EXEC sp_executesql @sql5;

-- Malicious payload:
DECLARE @input5_mal NVARCHAR(100) = 'Library'; DROP TABLE
Book_Recomandations; --';
DECLARE @sql5_mal NVARCHAR(MAX) =
        'SELECT * FROM Meetings WHERE location_place = '' +
        @input5_mal + ''';';
EXEC sp_executesql @sql5_mal;

```

Effect: Executes SELECT * FROM Meetings WHERE location_place = 'Library'; and then DROP TABLE Book_Recomandations;.

Prevention: Parameterize the location_place input so that it cannot break into a second statement:

```

DECLARE @input5_safe NVARCHAR(100) = 'Library'; DROP TABLE
Book_Recomandations; --';

DECLARE @sql5_safe NVARCHAR(MAX) = N'
        SELECT *
        FROM Meetings
        WHERE location_place = @loc;
';

EXEC sp_executesql
        @sql5_safe,
        N'@loc NVARCHAR(100)',
        @loc = @input5_safe;

```

II. Key Prevention Strategies (Applicable to All Cases):

1. **Parameterized Queries / Prepared Statements:** Always pass user input as parameters, never concatenate directly into SQL.
2. **Input Validation & Whitelisting:** Verify that inputs conform to a strict format (e.g., allowed column names) before including them in SQL.
3. **Use of Database-Specific Quotation Functions:** Functions like QUOTENAME() in SQL Server or proper placeholder syntax in other systems ensure identifiers and literals are safely quoted.

4. **Least Privilege Principle:** Grant applications only the minimum permissions needed. Even if an injection occurs, a restricted database user can't drop tables or access sensitive data.
5. **Stored Procedures Without Dynamic SQL:** When possible, implement logic in stored procedures that do not rely on dynamic SQL concatenation.
6. **Escaping or Rejecting Special Characters:** In contexts where parameterization isn't possible (rare), properly escape single quotes and other metacharacters or reject suspicious input patterns.